

Agenda for Today

- Computer hardware
- Computer software
- **helloworld.py → your first Python program!!**
- More on memory
- **magic_number.py → your second Python program**

Topics (I might blow through some of these pretty quickly)

- 1.1 Why Program?
- 1.2 Computer Systems: Hardware and Software
- 1.3 Programs and Programming Languages
- 1.4 What Is a Program Made of?
- 1.5 Input, Processing, and Output
- 1.6 The Programming Process

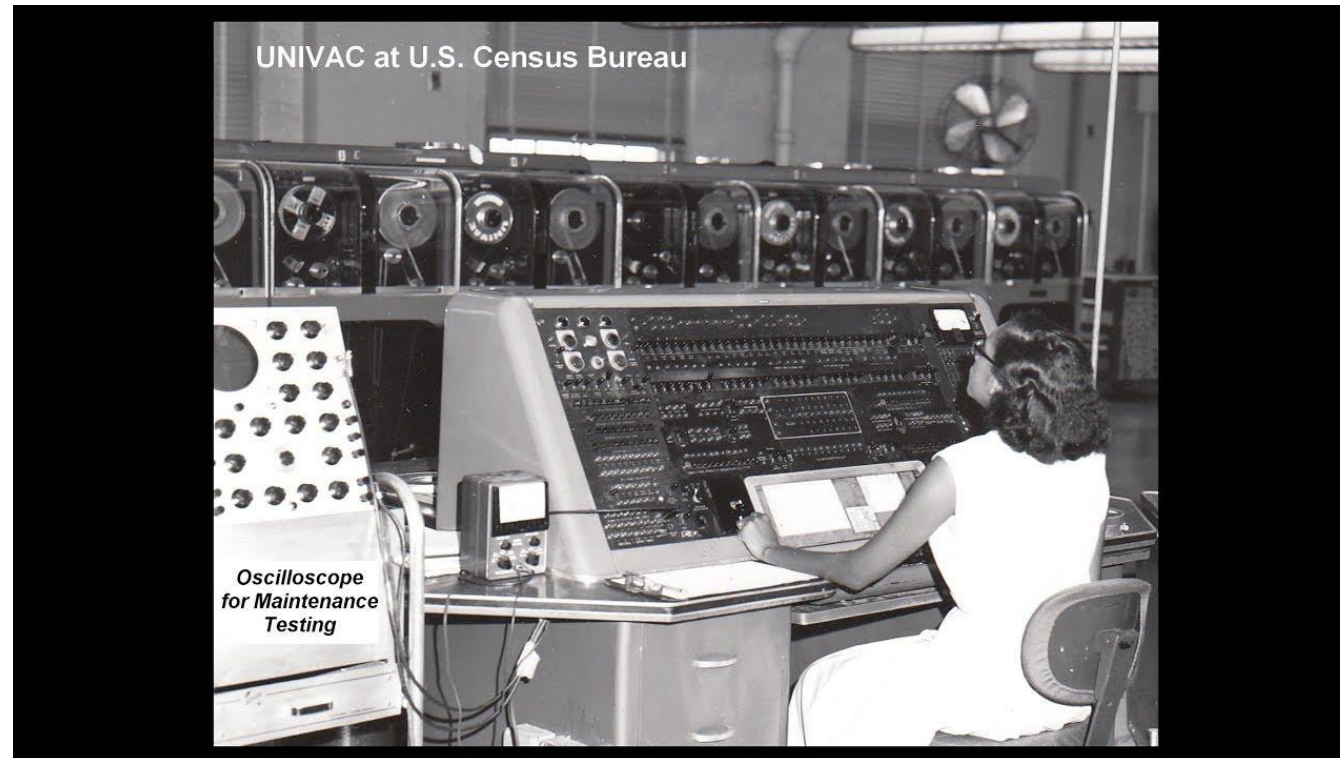
Why Program?

- **Computer** – programmable machine designed to follow instructions
- **Program / Software** – instructions that a computer follows to perform a task
- **Programmer** – person with the right skills who designs, creates, and tests programs for computers
- **So, without programmers, no programs; without programs, the computer cannot do anything.**

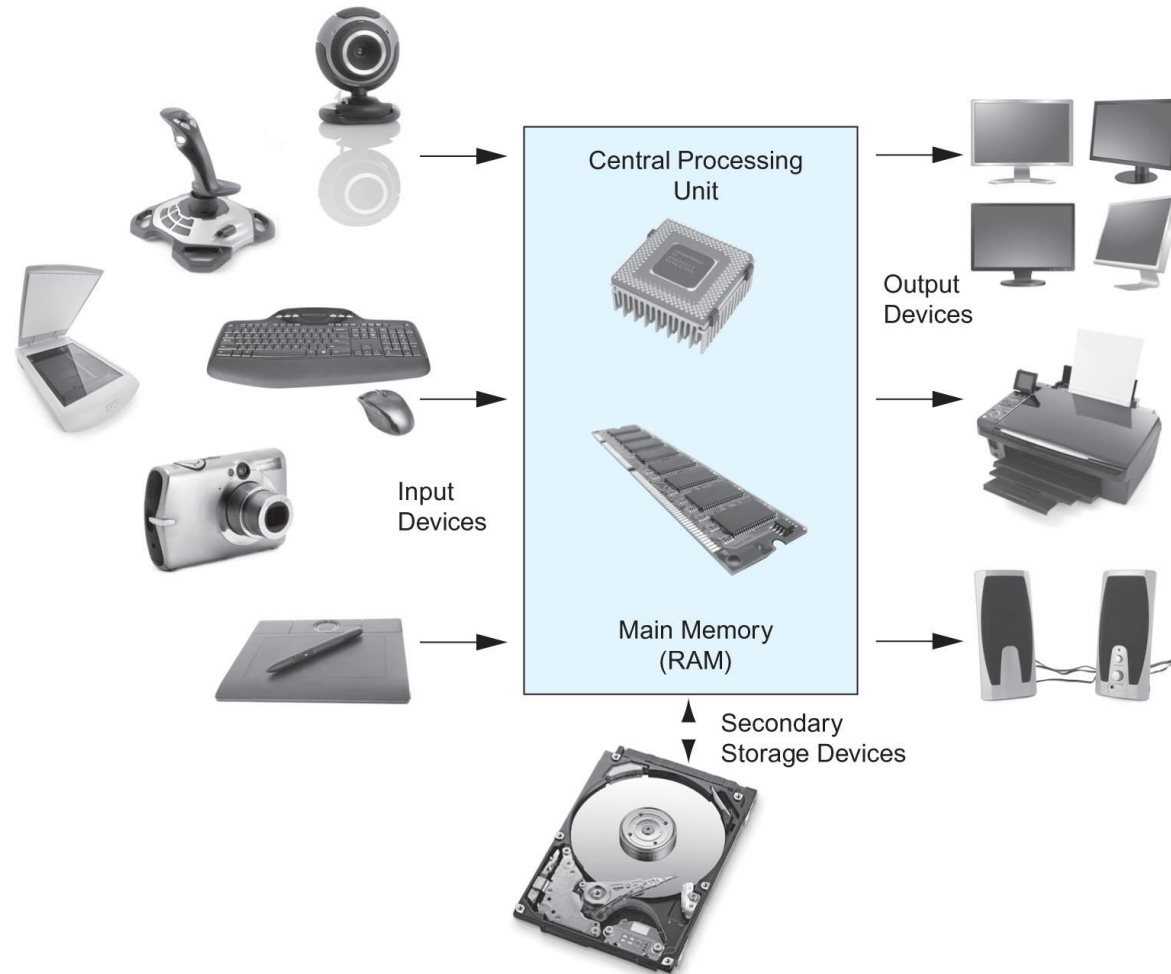
Hardware and Software

- **Hardware**
 - physical components of a computer
- Main Hardware Component Categories
 - Central Processing Unit (CPU)
 - Main memory (RAM)
 - Secondary storage devices
 - Input devices
 - Output devices

UNIVAC – First Commercial Computer



Main Hardware Component Categories

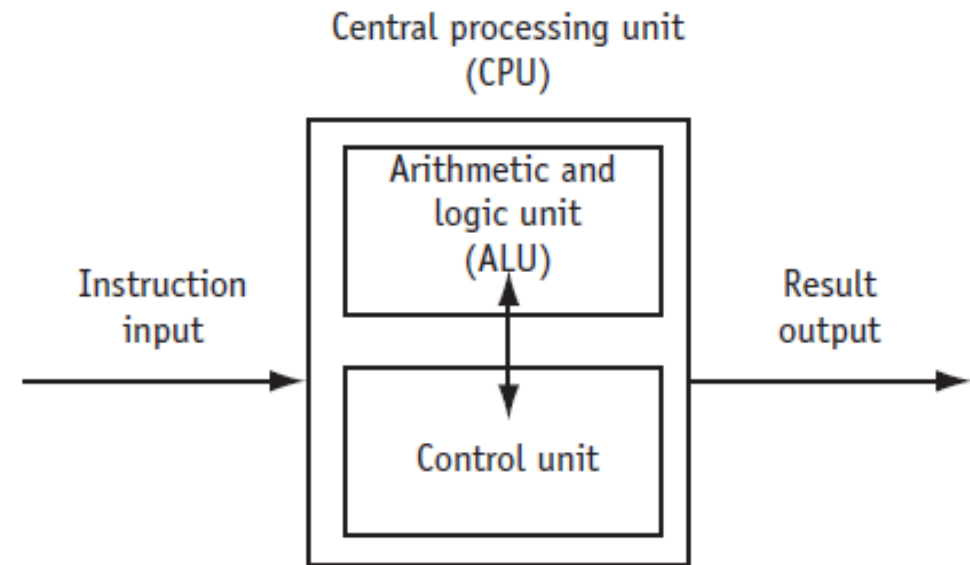


Central Processing Unit (CPU)

- CPU – hardware component that runs programs

Includes

- Control Unit
 - Retrieves and decodes program instructions
 - Coordinates computer operations
- Arithmetic & Logic Unit (ALU)
 - Performs mathematical operations



The CPU's Role in Running a Program

- Cycle through:

- **Fetch**

: get the next program instruction from main memory

- **Decode**

: interpret the instruction and generate a signal

- **Execute**

: route the signal to the appropriate component to perform an operation

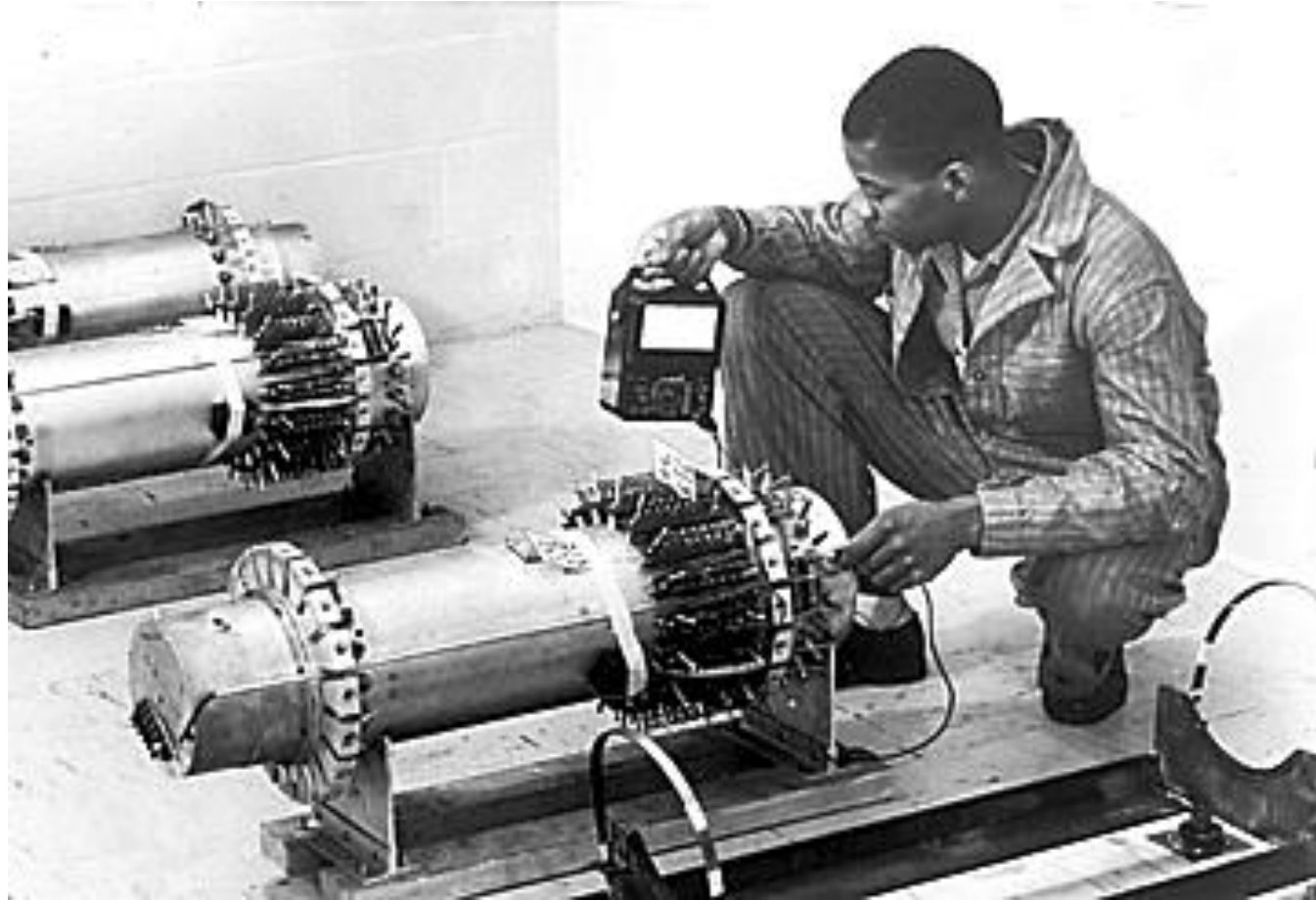
Main Memory

- Holds both program instructions and data
- Volatile – erased when the program terminates or the computer is turned off
- Also called Random Access Memory (RAM), because the CPU can access data and instructions from any memory location

Main Memory Organization

- **Bit**
 - Smallest piece of memory
 - Stands for binary digit
 - Holds an electrical charge
 - A positive charge is “on”
 - A negative charge is “off”
- **Byte**
 - Is 8 consecutive bits
 - Has an address in memory
 - There are millions (or even billions) of bytes of memory in a computer

Univac Memory (early 1950s)



Univac Memory (early 1950s)

- Each of these tubes contains mercury.
- If there's a wave in the mercury, it's a 1. If not, it's a 0.
- This was the most advanced computer in the world when it was built — and it stores less memory than a pocket calculator

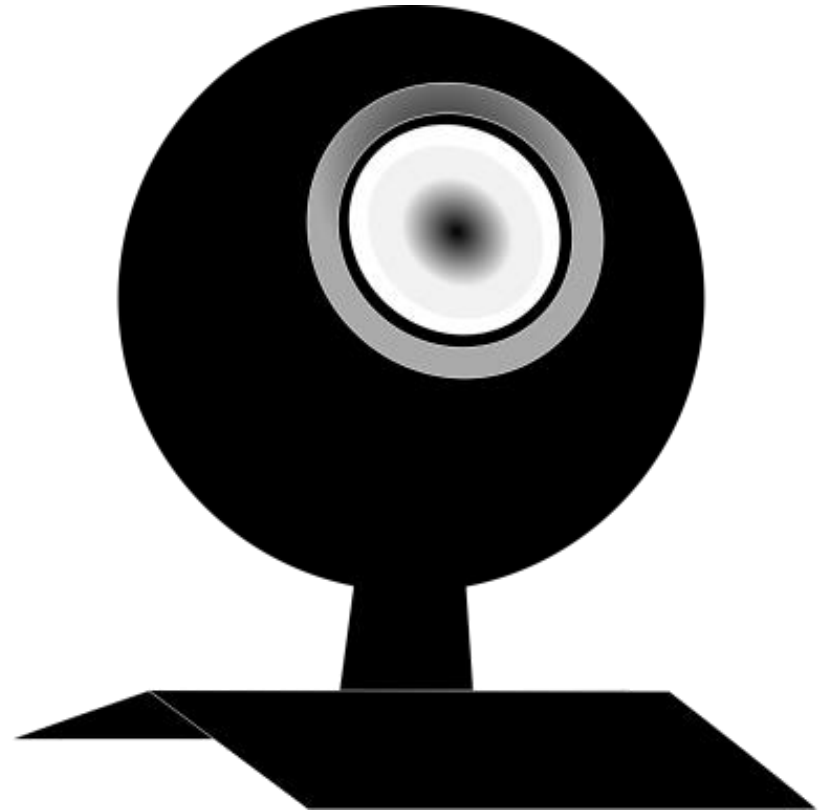


Secondary Storage

- Holds data when the program is not running or when the computer is turned off
- Several forms of secondary storage
 - Disk drive: mounted inside the computer or connected externally. Data is stored magnetically
 - Solid-state drive (SSD) / flash storage: no moving parts, much faster than a spinning disk
 - Optical: CD or DVD drive (increasingly obsolete)
 - USB flash drive

Input Devices

- Used to send information to the computer from outside
- Many devices can provide input: keyboard, mouse, touch screen, microphone, scanner, digital camera, disk drive, CD/DVD drive, USB flash drive



Output Devices

- Used to send information from the computer to the outside
- Many devices can be used for output: computer screen, printer, speakers, disk drive, CD/DVD recorder, USB flash drive



Software Programs That Run on a Computer

- **System software**
 - Programs that manage the computer hardware and the programs that run on the computer
- **Operating Systems**
 - Controls the operation of the computer
 - Manages connected devices and access to storage devices
 - Allows programs to run
- **Utility Programs**
 - Support programs that enhance computer operations
 - Examples: virus scanners, data backup, file compression
- **Software development tools**
 - Used by programmers to create software
 - Examples: text editors, interpreters, integrated development environments (IDEs)

1.3 Programs and Programming Languages

- **Program**
 - a set of instructions directing a computer to perform a task
- **Programming Language**
 - a special notation system used to write programs

Algorithm

- Algorithm: a set of well-defined steps to perform a task or to solve a problem
- Order is important. Steps must be performed sequentially.

Programs and Programming Languages

- Types of languages
 - Low-level: used for communication with computer hardware directly. Not very easy for a person to read.
 - High-level: closer to human language — this is where Python lives.

This is what assembly looks like:

```
_fib:
    pushl %ebp
    movl %esp, %ebp
    pushl %ebx
    pushl %ecx
    # Saved Registers

    movl 8(%ebp), %ecx      # ecx = n
    movl $0, %ebx          # ebx = secondlast = 1
    movl $1, %eax          # eax = last = 0
    jmp  fib2

fib1:
    add %ebx, %eax          # last = last + secondlast
    neg %ebx
    add %eax, %ebx          # secondlast = -secondlast + last
    dec %ecx                # n = n - 1

fib2:
    cmp $0, %ecx
    jne  fib1               # if n != 0 goto fib1

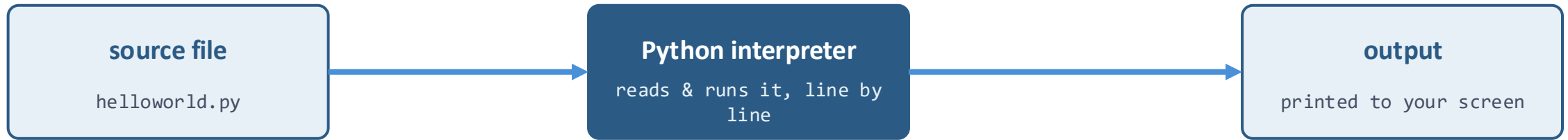
    # Restore registers and return
    popl %ecx
    popl %ebx
    popl %ebp
    ret
```

Take Systems Programming if you want to learn how to read this.

From a High-Level Program to a Running Program

- Create a file containing the program with a text editor.
 - program statements: source code
 - file: source file (for Python, this ends in .py)
- Some languages (like C++) need a separate compiler to translate the whole file into machine code before it can run.
- Python is different: it's an interpreted language.

How Python Runs Your Program

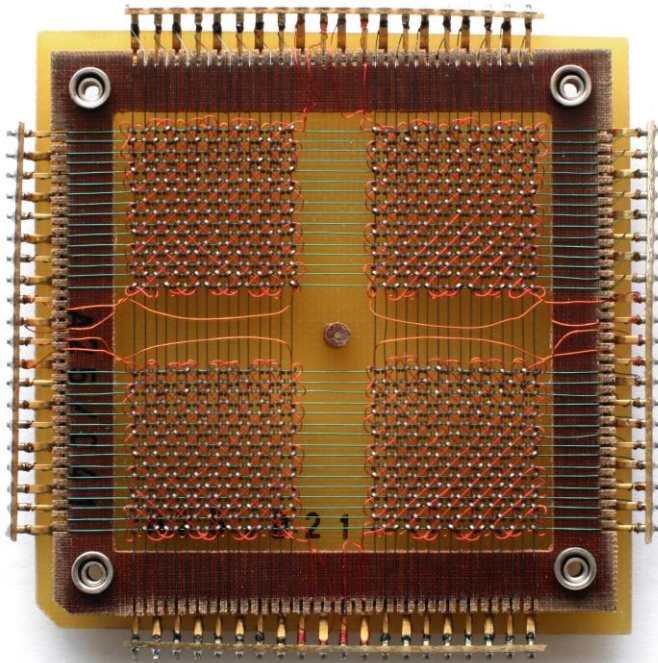


- No separate compiling or linking step — you just run the file and Python executes it immediately.
- This makes Python great for a first language: write a line, run it, see what happens.
- Errors that occur while running will stop the program at that point.

Putting It All Together

- There are many software development tools that provide an integrated development environment (IDE). An IDE has:
 - a text editor
 - an interpreter (to run your Python code)
 - a debugger
 - other supporting utilities
 - a user interface that ties them all together

Memory

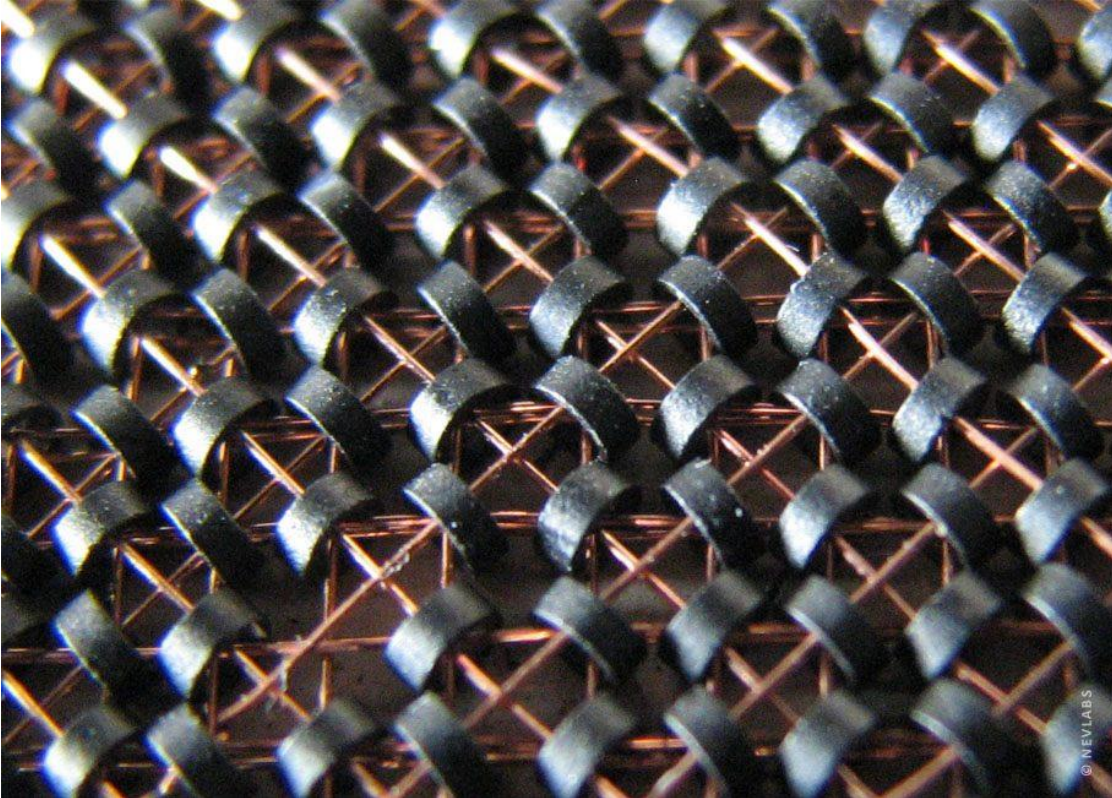


This is what memory looked like in 1970s computers — the last time memory was big enough to see with the naked eye.

The thing you will be doing most often when you write programs is writing to memory and reading from memory.

This board holds about 768 bits. (I counted!)

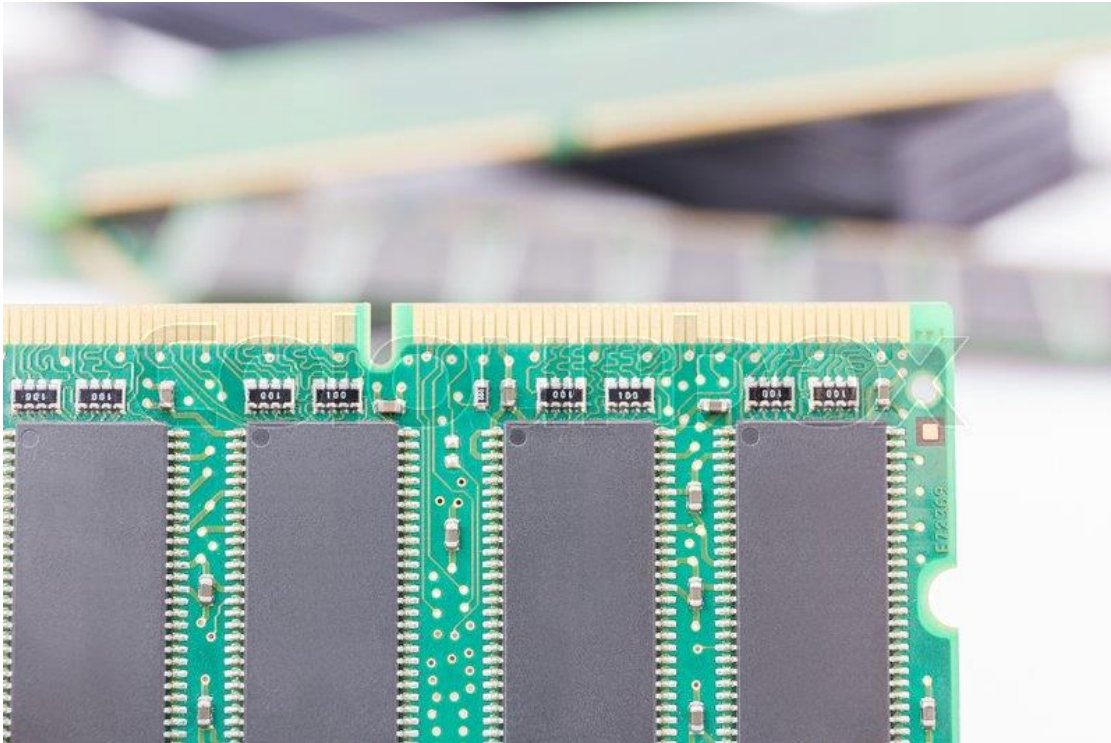
Memory



Here's a zoomed-in picture of 1970s memory — each one of these metal disks is a bit. It's a 1 if it's magnetized, and a 0 if it's not.

(this is about 72 bits)

Memory



Here's a modern memory chip — this board contains billions of bits, all of which are far too small to see.

Reading from memory and writing to memory is the thing we do most often when we write computer programs.

This board holds about 128,000,000,000 bits.

Memory – an Analogy

- You can think of your computer's memory as something like a gigantic spreadsheet.
- Each cell on the spreadsheet (or each address in memory) can hold a piece of data — a number, a string of letters, a true-or-false value, etc.

	A	B	C	D	E	F	G	H
1	STUDENT	MULTIPLE CHOICE	q1	q2	q3	q4	extra credit	TOTAL
2	NAMES	33	9	9	10	12	0	73
3	DELETED	33	15	14	15	15	5	97
4	BECAUSE	39	15	10	15	15	10	104
5	IT	33	13	8	15	15	5	89
6	WOULD	24	15	15	1	5	10	70
7	BE	24	7	0	12	14	5	62
8	RUDE	24	15	15	15	15	0	84
9	TO	30	10	15	15	15	0	85
10	SHOW	18	15	15	5	10	3	66
11	THEM	27	15	5	10	15	5	77
12	TO	36	15	15	15	15	10	106
13	YOU	30	14	15	15	14	0	88

Memory – an Analogy

- You can think of your computer's memory as something like a gigantic spreadsheet.
- Each cell on the spreadsheet (or each address in memory) can hold a piece of data — a number, a string of letters, a true-or-false value, etc.

	A	B	C	D	E	F	G	H
1	STUDENT	MULTIPLE CHOICE	q1	q2	q3	q4	extra credit	TOTAL
2	NAMES	33	9	9	10	12	0	73
3	DELETED	33	15	14	15	15	5	97
4	BECAUSE	39	15	10	15	15	10	104
5	IT	33	13	8	15	15	5	89
6	WOULD	24	15	15	1	5	10	70
7	BE	24	7	0	12	14	5	62
8	RUDE	24	15	15	15	15	0	84
9	TO	30	10	15	15	15	0	85
10	SHOW	18	15	15	5	10	3	66
11	THEM	27	15	5	10	15	5	77
12	TO	36	15	15	15	15	10	106
13	YOU	30	14	15	15	14	0	88

We can think of this as the string "hello" stored at address A2 on the spreadsheet.

Memory – an Analogy

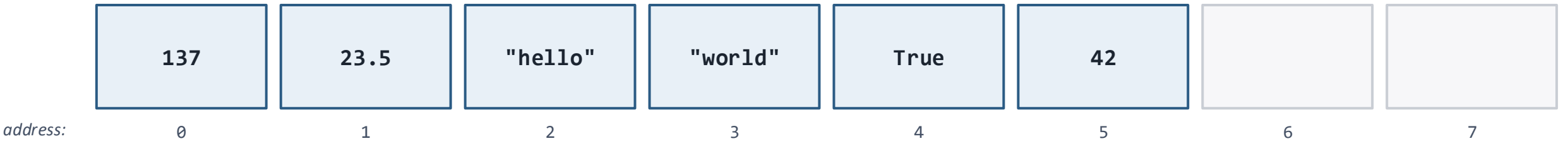
- You can think of your computer's memory as something like a gigantic spreadsheet.
- Each cell on the spreadsheet (or each address in memory) can hold a piece of data — a number, a string of letters, a true-or-false value, etc.

	A	B	C	D	E	F	G	H
1	STUDENT	MULTIPLE CHOICE	q1	q2	q3	q4	extra credit	TOTAL
2	NAMES	33	9	9	10	12	0	73
3	DELETED	33	15	14	15	15	5	97
4	BECAUSE	39	15	10	15	15	10	104
5	IT	33	13	8	15	15	5	89
6	WOULD	24	15	15	1	5	10	70
7	BE	24	7	0	12	14	5	62
8	RUDE	24	15	15	15	15	0	84
9	TO	30	10	15	15	15	0	85
10	SHOW	18	15	15	5	10	3	66
11	THEM	27	15	5	10	15	5	77
12	TO	36	15	15	15	15	10	106
13	YOU	30	14	15	15	14	0	88

We can think of this as the integer 30 stored at address H13 on the spreadsheet. (Booleans — true-or-false values — can live here too.)

Memory – Another Analogy

- A spreadsheet is two-dimensional ... but on your computer, memory addresses are all in a line.
- Think of memory as an extremely long strip of paper, divided into little boxes. Each box can hold one piece of data (a number, a string, a boolean value, etc.)
- The first box is at address 0, the second at address 1, and so on — all the way to however much memory your computer has.

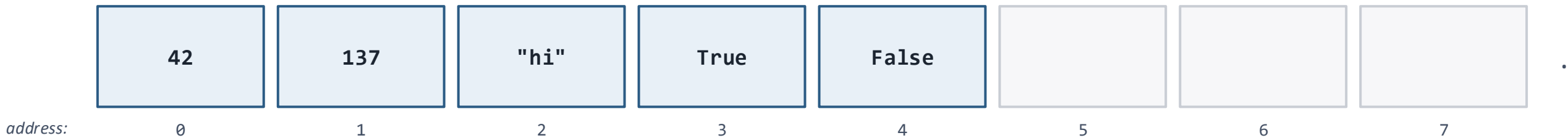


** this is a simplification for the purposes of this class — for the details, take Systems Programming*

From Bits to Boxes

Now you may be thinking:

- Bits are 0s and 1s
- Boxes hold numbers, letters, strings of letters ... how do you turn 0s and 1s into numbers and words?



How Binary Works

The trick is binary notation — counting in base two.

BASE TWO	BASE TEN
0	0
1	1
10	2
11	3
100	4
101	5
110	6
111	7
1000	8
1001	9
1010	10

You can represent the number ten with 4 bits.



How Binary Works

That's how you can represent numbers like 137 or 42 in a computer — by using bits to write down the base-two version.

BASE TWO	BASE TEN
0	0
1	1
10	2
11	3
100	4
101	5
110	6
111	7
1000	8
1001	9
1010	10

How Binary Works — Quick Quiz

- You can represent 8 different numbers with 3 bits (every number from 0 to 7).
- **How many different numbers can you represent with four bits?**
- **What about five bits?**
- (spend a minute or so working it out on paper if you need to)

From Numbers to Letters

- Okay, that's how we represent numbers. But how do we represent letters and words?

From Numbers to Letters

Dec	Char	Dec	Char
64	@	96	~
65	A	97	a
66	B	98	b
67	C	99	c
68	D	100	d
69	E	101	e
70	F	102	f
71	G	103	g
72	H	104	h
73	I	105	i
74	J	106	j
75	K	107	k
76	L	108	l
77	M	109	m
78	N	110	n
79	O	111	o
80	P	112	p
81	Q	113	q
82	R	114	r
83	S	115	s
84	T	116	t
85	U	117	u
86	V	118	v
87	W	119	w
88	X	120	x
89	Y	121	y
90	Z	122	z
91	[	123	{
92	\	124	
93	]	125	}
94	^	126	~
--		---	

Here's the trick: there's an arbitrary standard called ASCII (“American Standard Code for Information Interchange”) that tells how to translate letters to numbers.

ASCII uses 7 bits to store each letter, plus 1 more bit for error detection. How many different characters can you represent in 7 bits?

8 bits is called a byte. A single character is 1 byte. A word (or sentence, or paragraph, or novel...) is just a bunch of characters put together in a string.

The number 65 means capital A. The number 113 means lowercase q.

What About 你好 and مرحبا and こんにちは ?



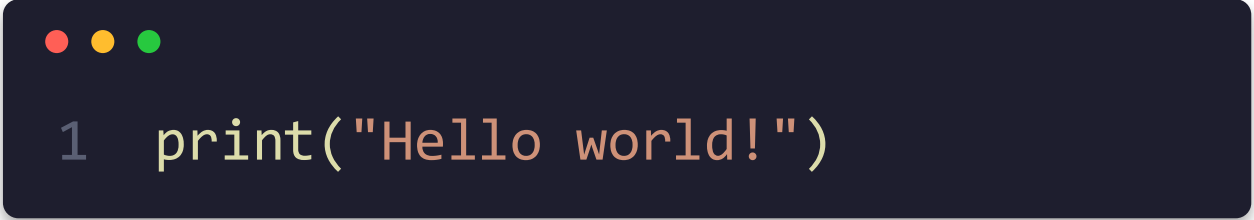
- ASCII (8 bits) can only represent characters in the Latin alphabet.
- What about all the other character sets in the world?
- Unicode is a character encoding that uses up to 32 bits (4 bytes) per character — enough for 4,294,967,296 different characters.
- (Nearly) every language in the world can be represented in Unicode.
- **Python strings are Unicode by default — you can use emoji, accents, and non-Latin scripts right out of the box.**

Python!

let's put this stuff to use

Hello World: Your First Python Program

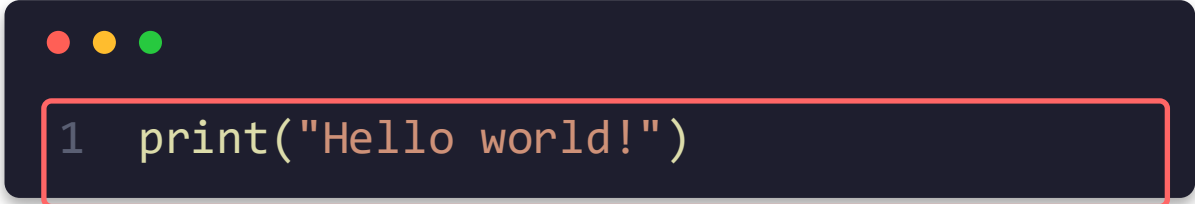
print() will show up in almost every program you write this semester.



```
1 print("Hello world!")
```

Hello World: No Setup Required

Python just runs your file from the top line to the bottom line.

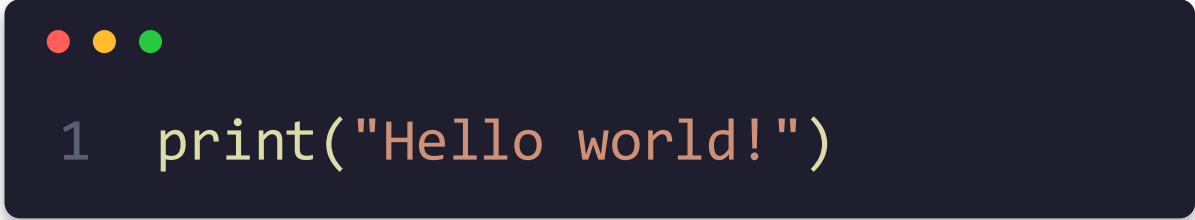


```
1 print("Hello world!")
```


Hello World: print(), Piece by Piece

`print` is a built-in function — a ready-made command that Python provides.

The parentheses () are what actually call the function — they say “do this now.”



```
1 print("Hello world!")
```

Hello World: the String Argument

Whatever is inside the parentheses is called an **argument** — it's what gets passed to the function. The quotation marks tell Python “this is text,” not a number or a command. Text like this is called a **string**. So this line means: print the string “Hello world!” to the console.



```
1 print("Hello world!")
```

Hello World: Running It



```
1 print("Hello world!")
```

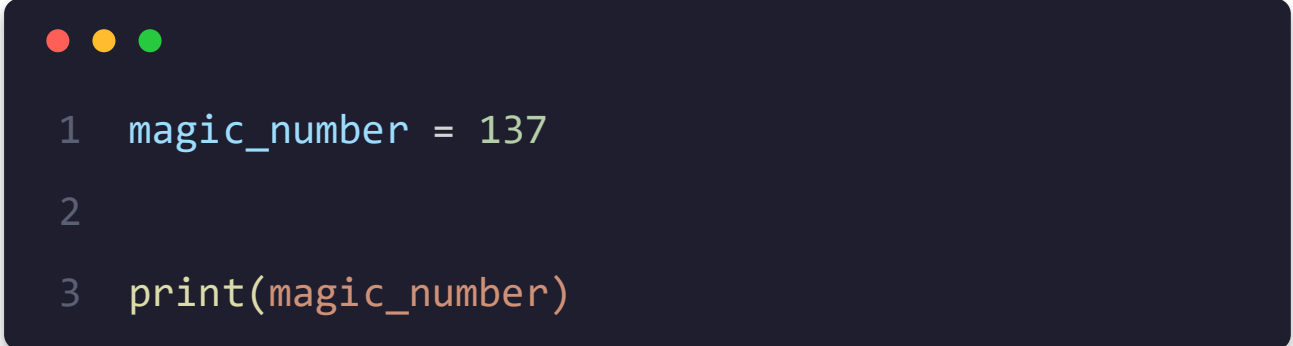


```
1 $ python3 helloworld.py  
2 Hello world!
```

Here's what I get when I run this:

Sample Program

Here's a sample program. We're going to look at it piece by piece.

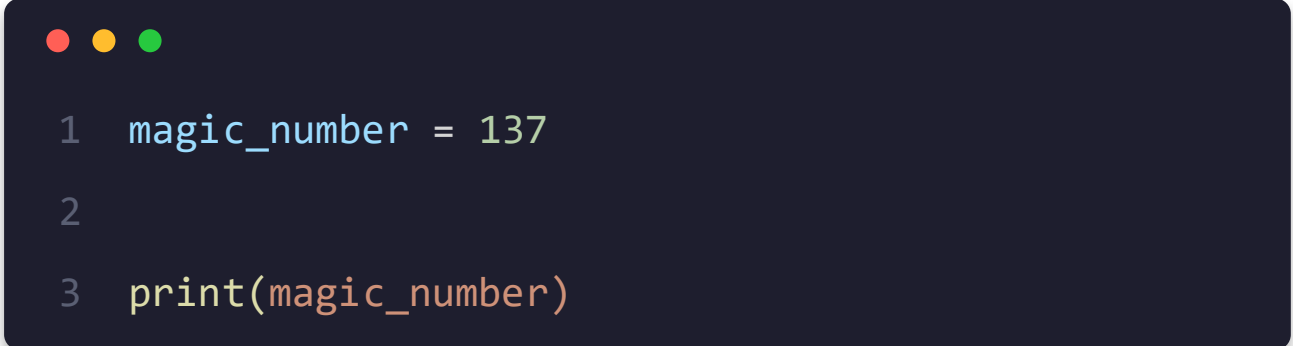


```
1 magic_number = 137
2
3 print(magic_number)
```

Sample Program — No Imports Needed

Some languages need you to “include” a library before you can print anything.

In Python, `print()` is just built in — nothing to import, nothing to set up.

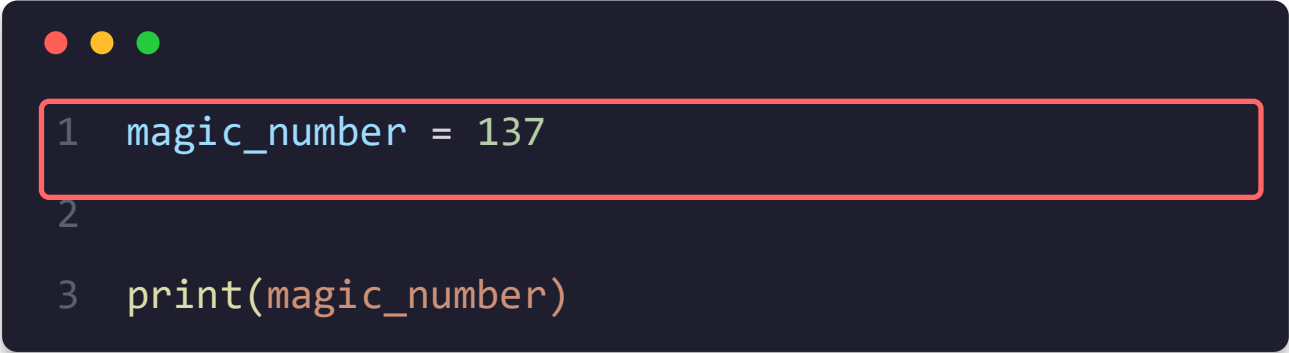


```
1 magic_number = 137
2
3 print(magic_number)
```

Sample Program — Where It Starts

There's no `main()` function here.

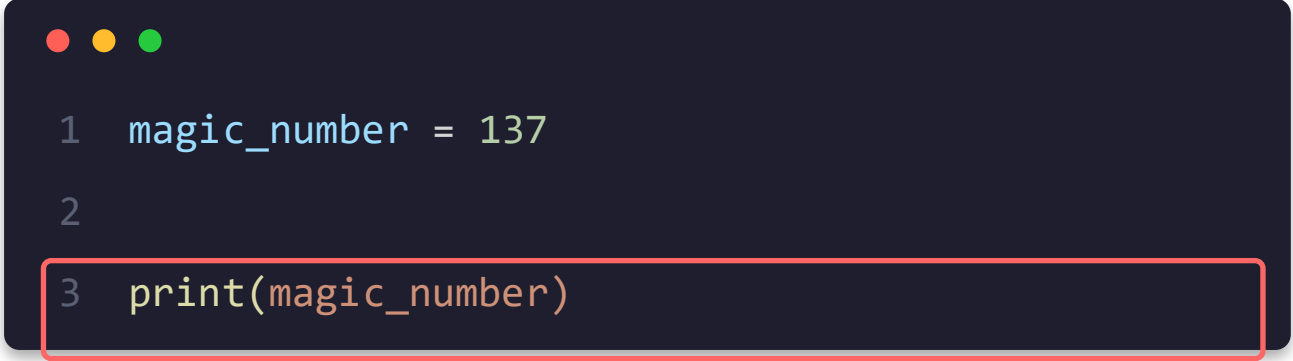
Python starts at the first line of the file and runs straight down — just like Karel's `main()` told the robot where to start, except here you don't have to write it.



```
1 magic_number = 137
2
3 print(magic_number)
```

Sample Program — Printing to the Console

You print to the console by calling `print()` with what you want to show.

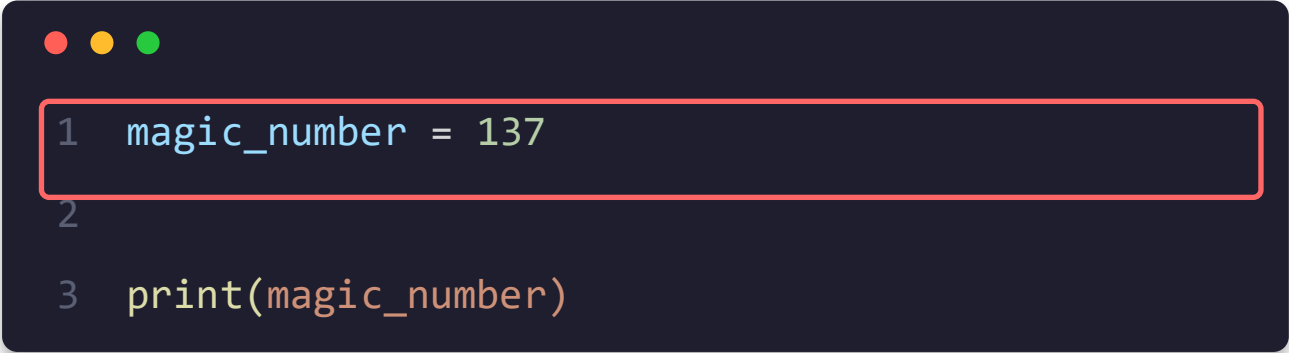


```
1 magic_number = 137
2
3 print(magic_number)
```

Sample Program — Creating a Variable

And here we create a variable.

(this is what I want to focus on right now)



```
1 magic_number = 137
2
3 print(magic_number)
```


Sample Program — What Does This Actually Do?

- Karel (deliberately) doesn't include variables. (That's part of what makes the puzzles interesting.)
 - `magic_number = 137`
- This line means: set aside space in memory, put the value 137 there, and let us call that space `magic_number`.
- **Notice we never had to say what type `magic_number` is. Python figures that out automatically from the value — 137 is a whole number, so `magic_number` becomes an integer.**

Sample Program — Finding a Home in Memory

```
1 magic_number = 137
2
3 print(magic_number)
```

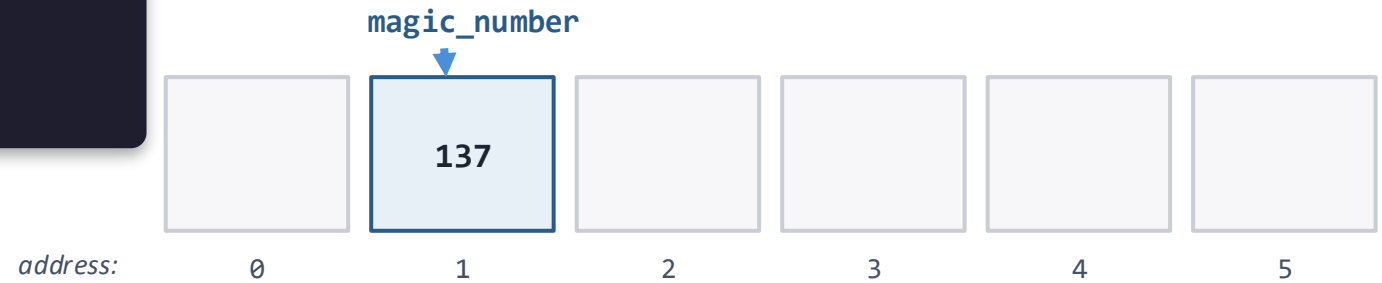


done!

When Python runs this line, it grabs a box that isn't storing anything yet and puts the number 137 there.

Sample Program — Naming the Box

```
1 magic_number = 137
2
3 print(magic_number)
```



It would be inconvenient to refer to this variable by its address, so we give it a name: `magic_number`. From now on, using the name `magic_number` means “go get the value in that box.”

Syntax Note — No Type Keyword

Some languages make you write the variable's type before its name, like:

```
int magic_number = 137;
```

Python skips that. You just write name = value, and Python figures out the type from the value on the right.

This is called dynamic typing.



```
1 magic_number = 137
```

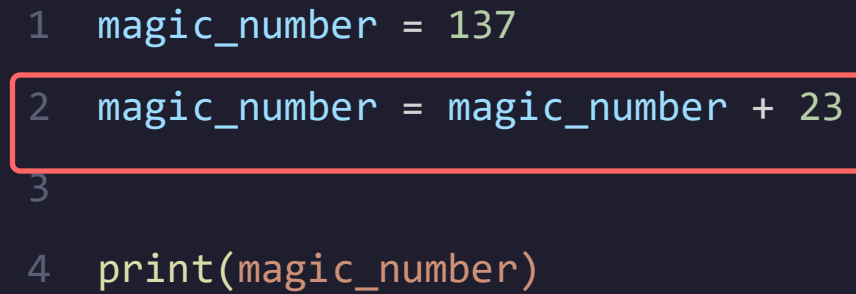
```
2
```

```
3 print(magic_number)
```

Doing Arithmetic

Here we do arithmetic on our stored variable.

This line means: get the value of `magic_number`, add 23 to it, then store the result back into `magic_number`.

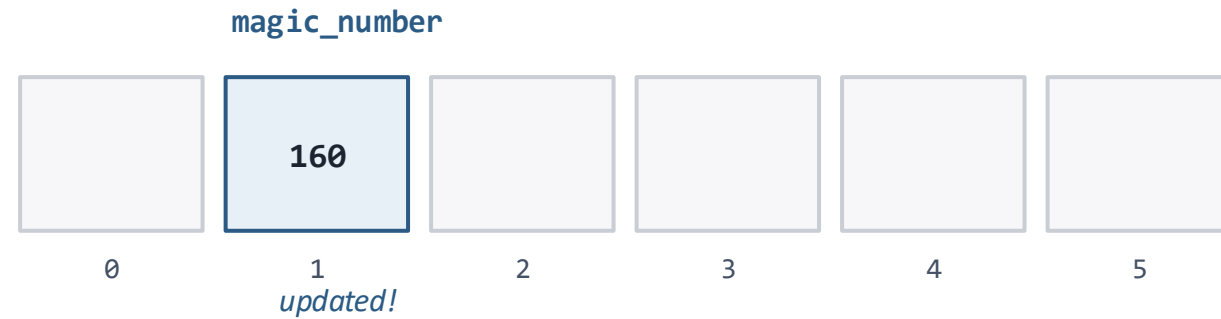
A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. It contains four lines of Python code. The second line, `magic_number = magic_number + 23`, is highlighted with a red rectangular border. The code is as follows:

```
1  magic_number = 137
2  magic_number = magic_number + 23
3
4  print(magic_number)
```

Doing Arithmetic — the Result

```
1 magic_number = 137
2 magic_number = magic_number + 23
3
4 print(magic_number)
```

address:



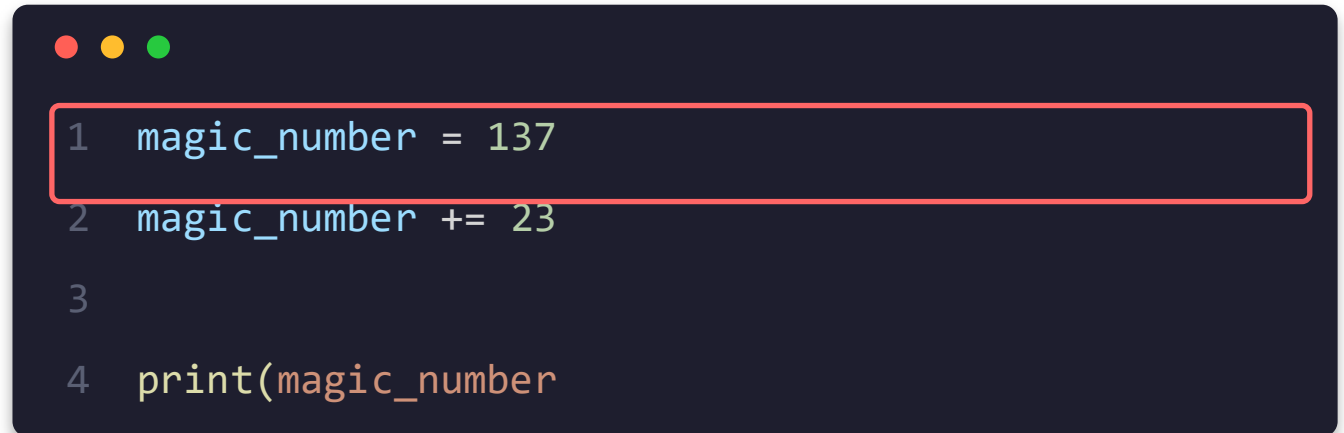
Syntax Note — Shorthand

Programmers do “add and store back” so often that Python has special syntax for it:

`magic_number += 23`

is exactly the same as

`magic_number = magic_number + 23`



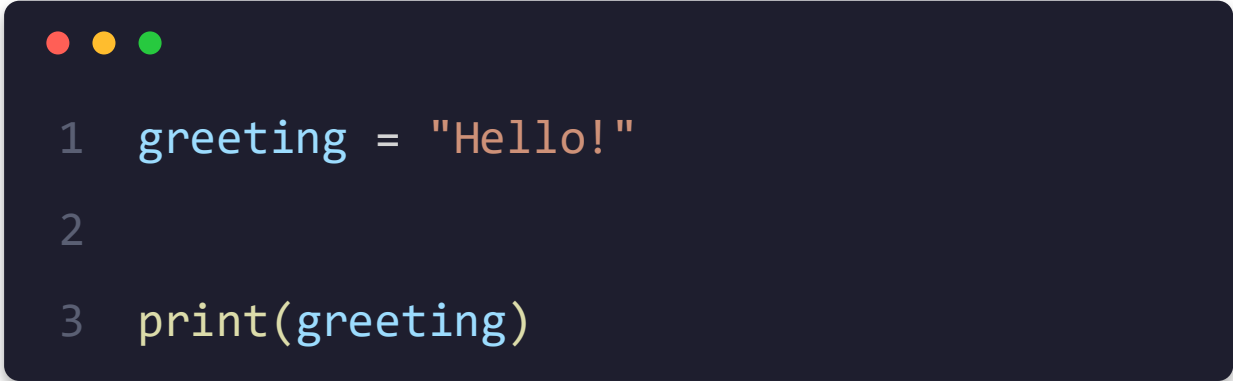
```
1 magic_number = 137
2 magic_number += 23
3
4 print(magic_number)
```

Sample Program — What About Text?

Okay, but what if we want to store whole words?

Some languages have a separate char type for a single letter, and you need to include a string library to store whole words.

Python doesn't bother with that distinction — strings are built in from the start, and a single character is just a string of length 1.



```
1  greeting = "Hello!"  
2  
3  print(greeting)
```